

Technical Roadmap

Date: 2026-06-29

Current Technical Baseline

The repository is a Python modular monolith with:

- FastAPI backend.
- SQLAlchemy models and services.
- Alembic migrations.
- Postgres database.
- Pydantic 2 schemas.
- APScheduler background scheduling.
- Reflex frontend prototype.
- pytest test suite using a Postgres test database.

The current state is useful but not yet clean enough for sustained product development.

Current Critical Issues

Highest-priority findings:

- The active Reflex UI is untracked while the old UI path is tracked as deleted.
- Key dependency/config files are ignored.
- `requirements.txt` does not describe the real runtime environment.
- `Dockerfile` is empty.
- Tests currently fail in recurring scheduling.
- Account creation does not create an owner membership.
- Account onboarding seeds recurring templates with emoji strings where category UUIDs are expected.
- Some search and summary filters are accepted by schemas but ignored in services.
- Header-only `X-User-Id` auth is unsafe.
- Several global list/read endpoints are too permissive.
- Financial profile schema includes fields not persisted in the DB model.
- The frontend is a temporary dashboard, not yet the product UX.

Technical Principles

Use a modular monolith until the product validates.

Prefer:

- Clear domain boundaries.
- Strong tests.
- Explicit authorization.
- Background jobs with auditability.

- Structured data models.
- Reproducible local setup.
- Explainable AI workflows.

Avoid for now:

- Premature microservices.
- Complex agent orchestration before basic workflows are stable.
- Bank aggregation before CSV/manual validation.
- Unbounded autonomous actions.

Target Architecture

Backend Modules

Identity and access:

- Users.
- Authentication.
- Sessions/tokens.
- Roles and permissions.
- Audit logs.

Accounts and memberships:

- Personal accounts.
- Shared accounts.
- Organizations later.
- Membership roles.
- Contribution shares.

Transactions:

- Expenses.
- Income later.
- Categories.
- Recurring templates.
- Currency normalization.
- Search and filters.

Financial planning:

- Financial profile.
- Budgets.
- Goals.
- Tax reserve.
- Safe-to-spend.
- Runway.

Insights and agents:

- Rules engine.
- Insight generation.
- Scheduled pattern detection.
- Recommendation impact simulations.
- Recommendation queue.
- Action inbox.
- Agent action plans.
- Approval workflows.
- Evaluation data.

Professional intelligence:

- Deadline calendar.
- Obligation templates by user type and jurisdiction.
- Funding opportunity watchlist.
- Regulatory/law-change watchlist integration later.
- Source, applicability, and confidence metadata.

Documents and integrations:

- Receipts.
- Invoices.
- CSV imports.
- Exports.
- Bank/accounting integrations later.

Notifications:

- Email.
- Push later.
- Scheduled reports.
- Alerts.
- Deadline reminders.
- Recommendation digests.

Data Flow

Recommended flow:

1. User imports, uploads, or manually enters financial data.
2. Transaction normalization standardizes amount, date, currency, account, category, and source.
3. Rules and models enrich the data.
4. Scheduled jobs detect patterns, risks, deadlines, and opportunities.
5. Insight engine produces explainable recommendations with evidence and impact estimates.
6. Action inbox presents recommendations.
7. User approves, dismisses, snoozes, or edits.

8. The system records feedback and audit events.

Pattern And Recommendation Engine

The first production-grade intelligence layer should be a deterministic service that runs on a schedule and creates insight records. AI can summarize and personalize explanations, but calculations should remain transparent.

Core inputs:

- Normalized transactions.
- Recurring templates.
- Financial profile.
- Goals and tax reserve settings.
- User feedback on previous insights.
- Professional deadline calendar.

Core outputs:

- Insight record.
- Evidence payload with transaction IDs and date ranges.
- Impact estimate, such as monthly savings, runway change, goal-date shift, or tax-reserve gap.
- Recommendation options.
- Confidence score.
- Notification eligibility.
- Audit log entry.

Example rules:

- Repeated weekday or weekend spending pattern.
- Category spend drift versus prior baseline.
- Subscription growth.
- Upcoming cash-flow pinch.
- Tax reserve underfunding.
- Recurring professional deadline approaching.

Design constraints:

- Make insights explainable and reproducible.
- Store enough evidence to debug why an insight was generated.
- Throttle repeated alerts.
- Let users mute categories, rules, or specific recommendations.
- Track accepted, dismissed, snoozed, and irrelevant feedback.
- Treat legal, tax, and funding alerts as informational until reviewed by a human or trusted professional source.

Professional Monitoring Architecture

Professional deadlines and external opportunity/law monitoring should start inside the monolith as a bounded module, then be extracted only if it proves valuable as a separate service.

Start with:

- Manual deadline templates for tax, VAT, insurance, invoice follow-up, and accounting tasks.
- User-configured jurisdiction, profession, business type, and filing cadence.
- Scheduled reminder generation.
- Source links for any external obligation template.

Later add:

- Funding/opportunity source ingestion.
- Law/regulatory source ingestion.
- Applicability matching against a professional profile.
- Human review or curation workflow for high-risk alerts.
- Versioned source snapshots and citation storage.
- API boundary so a future separate monitoring service can feed this product.

Do not begin with broad legal monitoring. It is a high-trust, high-liability surface. Validate with narrow deadline reminders and curated funding alerts first.

Roadmap

Phase T0: Clean Repo Baseline

Objective: Make the repository reproducible and safe to build on.

Tasks:

- Track dependency files.
- Decide whether active Reflex UI replaces old tracked UI.
- Add root README.
- Add `.env.example`.
- Replace empty Dockerfile or remove it until useful.
- Clean `.gitignore`.
- Remove or quarantine temp/archive files.
- Add setup and test commands.
- Ensure `pytest` passes.

Exit criteria:

- Fresh clone can install dependencies and run tests.
- `git status` only shows intentional work.
- Active app files are tracked.

Phase T1: Correctness And Data Model Stabilization

Objective: Fix known correctness issues before adding features.

Tasks:

- Fix account creation to create owner membership.

- Fix category seeding and onboarding.
- Fix recurring scheduler to process all due templates.
- Fix expense search filters.
- Fix budget summary filters.
- Fix membership share update.
- Align financial profile schema and DB model.
- Add migration for missing fields or remove unused fields.
- Add tests for account creation, onboarding, and recurring catch-up.

Exit criteria:

- Tests pass.
- Main manual flows work end to end.
- API behavior matches schema promises.

Phase T2: Security And Authorization

Objective: Replace prototype auth and close data leaks.

Tasks:

- Add real authentication.
- Replace `x-User-Id` pseudo-auth.
- Add passwordless email, OAuth, or simple JWT auth for MVP.
- Add account-scoped authorization helpers.
- Remove global list endpoints or restrict to admin.
- Enforce VIEWER read-only server-side.
- Add CORS configuration.
- Add audit logs for sensitive actions.
- Add tests for authorization boundaries.

Exit criteria:

- Users cannot access accounts, memberships, expenses, or profiles outside their scope.
- Role behavior is enforced in backend, not just UI.

Phase T3: Frontend Product Shell

Objective: Replace the temporary dashboard feel with a product-oriented UI.

Tasks:

- Create Today page.
- Move account snapshot into a decision-focused layout.
- Add Action Inbox.
- Improve transaction table.
- Add cash-flow timeline.
- Add clear empty states.
- Remove emoji-heavy developer styling.

- Add mobile-responsive layout.
- Decide whether Reflex remains long-term frontend.

Exit criteria:

- First screen communicates product value.
- A pilot user can understand what action to take next.

Phase T4: Import, Export, And Documents

Objective: Reduce manual data entry.

Tasks:

- CSV import.
- Import review screen.
- Duplicate detection.
- CSV/PDF export.
- Receipt attachment model.
- File storage abstraction.
- OCR proof of concept.

Exit criteria:

- Users can bring real financial data into the product.
- Export supports practical tax/accounting workflows.

Phase T5: Insight Engine

Objective: Generate useful recommendations without overusing AI.

Tasks:

- Create insight table/model.
- Create recommendation queue.
- Add rule-based insights.
- Add safe-to-spend v0.
- Add tax reserve v0.
- Add upcoming bill warnings.
- Add repeated spending pattern detection.
- Add recommendation impact simulation.
- Add insight evidence payloads.
- Add snooze/dismiss/accepted feedback states.
- Add weekly summary.
- Track user feedback on insights.
- Add scheduled insight generation.
- Add deadline reminder model.

Exit criteria:

- Product provides actionable recommendations from real data.

- Insight precision can be measured.
- Users can understand why a recommendation was generated.
- Users can control notification noise.

Phase T6: Professional Intelligence

Objective: Add deadline and opportunity monitoring for professionals without overbuilding legal intelligence.

Tasks:

- Create professional profile fields: jurisdiction, work type, business type, filing cadence.
- Add deadline templates for tax, VAT, social insurance, accounting, invoice follow-up, and contract renewals.
- Add reminder schedules and escalation rules.
- Add source metadata and applicability assumptions.
- Add funding opportunity watchlist prototype.
- Add fake-door or concierge workflow for law-change alerts.
- Define API contract for a future separate regulatory/funding monitoring service.

Exit criteria:

- Pilot professionals receive useful deadline reminders.
- Funding/law-change alerts are validated before automated ingestion.
- The system remains informational and avoids unsafe legal/tax claims.

Phase T7: AI And Agent Layer

Objective: Add bounded, explainable AI workflows.

Tasks:

- Build categorization evaluation dataset.
- Add LLM-assisted categorization with confidence threshold.
- Add explanation generation.
- Add document extraction evaluation.
- Add agent action schema.
- Add approval policy engine.
- Add action ledger.
- Add rollback/undo support where possible.

Exit criteria:

- Agents produce bounded recommendations with evidence.
- Risky actions require explicit approval.
- AI quality is measured before launch.

Phase T8: Scale And Service Boundaries

Objective: Prepare for business expansion without premature microservices.

Tasks:

- Move background jobs to a worker queue.
- Add observability.
- Add structured logs and metrics.
- Add rate limiting.
- Add data retention policies.
- Add integration health checks.
- Identify service extraction candidates.

Microservices should be considered only when:

- Independent scaling is needed.
- Team ownership requires it.
- Integrations/jobs create operational isolation needs.
- The monolith becomes a deployment bottleneck.

Likely future service boundaries:

- Identity.
- Transactions.
- Documents.
- Insights/agents.
- Notifications.
- Integrations.
- Professional monitoring, if funding/law-change coverage becomes a reusable service.

Testing Strategy

Short term:

- Keep Postgres-based integration tests.
- Add service-level unit tests for financial logic.
- Add API authorization tests.
- Add migration tests where feasible.

Medium term:

- Add frontend smoke tests.
- Add import/export golden files.
- Add insight evaluation fixtures.
- Add scheduled insight fixture tests for repeated behavior and deadline reminders.
- Add AI regression tests.

Long term:

- Add end-to-end tests for approved agent actions.
- Add monitoring-driven quality checks.

Infrastructure Strategy

Short term:

- Local Postgres.
- Alembic migrations.
- Manual dev server.
- Clear setup docs.

Medium term:

- Docker Compose.
- CI tests.
- Hosted app environment.
- Managed Postgres.
- Object storage for documents.
- Worker process.

Long term:

- Queue and scheduled workers.
- Metrics/logging/tracing.
- Secrets manager.
- Backup/restore.
- Environment-specific deployments.

Security And Governance

Required before real users:

- Real auth.
- Authorization by account and role.
- Secure secrets.
- PII handling.
- Data deletion/export.
- Audit logs.
- Least-privilege integrations.

Required before agents:

- Approval policy engine.
- Action ledger.
- Evidence and explanation for recommendations.
- Human confirmation for risky actions.
- Failure handling and rollback.

Required before professional monitoring:

- Source provenance.
- Jurisdiction and applicability metadata.
- Alert confidence and freshness checks.

- Clear non-advice disclaimers.
- Review workflow for high-risk external alerts.

Recommended Immediate Milestone

Milestone: Clean Build Base

Tasks:

- Fix account creation and recurring scheduling.
- Fix repo tracking and dependency files.
- Add root README and `.env.example`.
- Make tests pass.
- Commit the active UI intentionally.
- List remaining debt as issues or backlog items.

After this milestone, product development can resume without carrying fragile foundations.